

Spring Boot

23 August 2026 10:33

What problem does spring boot solve ?

In traditional spring application, developer had to configure many thing manually

Problems:

1. To much configuration
2. Dependency version management
3. Boilerplate code
4. Difficult for beginners
5. Long project setup

Spring boot solve these problem using :

1. Auto-configuration
2. Starter dependencies
3. Embedded server
4. Production-ready features

Spring boot = Spring - XML configuration + Auto configuration

Spring boot = Spring + auto setup + embedded server + starters

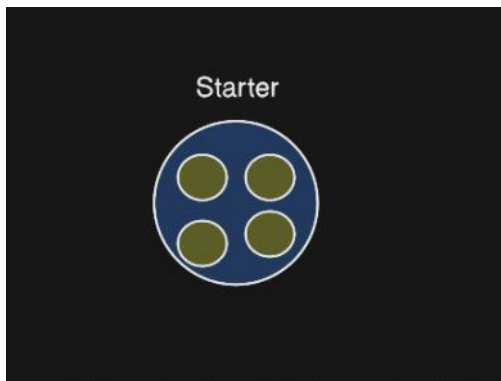
Note: Spring boot main job is to reduce configuration and help is to start quickly.

Note: Spring boot internally used spring. All spring core concepts still apply.

Important Spring boot Features:

1. Auto-config
2. Starters
3. Embedded server
4. Spring boot CLI
5. Actuator
6. Easy REST API dev
7. Easy to integrate with DB

What is Starter dependencies ?



```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

Common Starters

- **spring-boot-starter-web**: Builds web and RESTful applications using Spring MVC and embedded Tomcat.
- **spring-boot-starter-data-jpa**: Connects to relational databases using Spring Data JPA and Hibernate.
- **spring-boot-starter-security**: Adds authentication and authorization features.
- **spring-boot-starter-test**: Provides testing tools like JUnit, Mockito, and AssertJ.
- **spring-boot-starter-actuator**: Adds production-ready monitoring and health checks.

spring-boot-starter-web:

This single jar(starter dependency) contains multiple jar

1. Spring MVC
2. Embedded tomcat
3. Jackson JSON library
4. Validation/web support
5. Logging support

Note: Starter means one dependency that brings a group of related dependencies. All the class inside these dependencies will be treated as spring bean automatically.

Embedded Server:

IN traditional java web apps we create jar/war and we deploy it into external tomcat server.

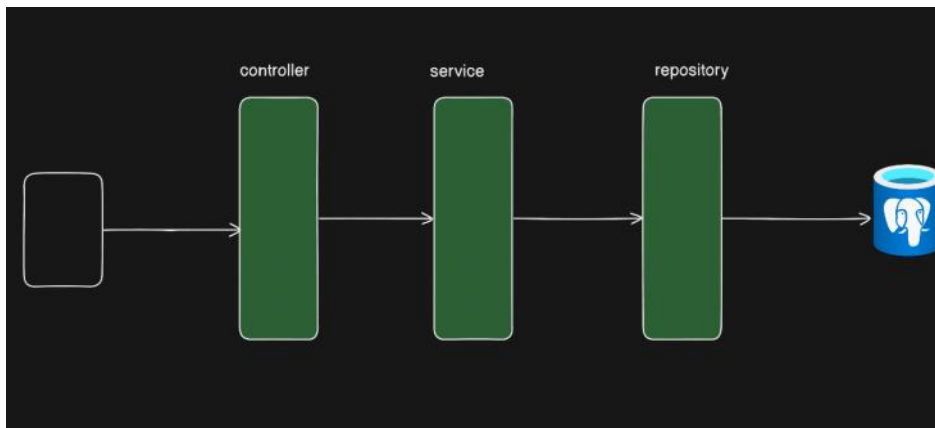
In spring boot tomcat is inside the application we run application directly.

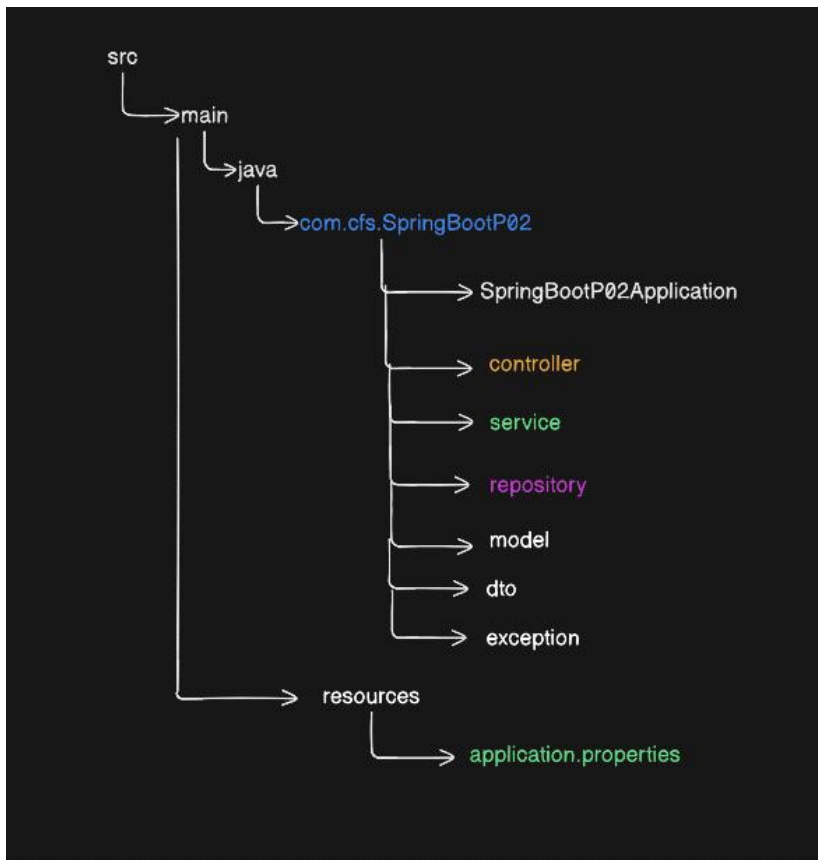
This makes deployment easier, each service can run independently on there dedicated servers.

How to create spring boot application

1. Spring initializer
2. IDE

Spring boot project structure





Note: Good package structure keep code readable as project grow.

Backend layering:

Controller -> receives request

Service -> Business logic

Repository --> database logic

Model/entity --> data object

DTO -> request /response object

Exception -> Error handling

Main class:

```
package com.cfs.SpringBootP02;
```

```
import org.springframework.boot.SpringApplication;
```

```
import org.springframework.boot.autoconfigure.SpringBootApplication;
```

```
@SpringBootApplication
```

```
public class SpringBootP02Application {
```

```
    public static void main(String[] args) {
```

```
        SpringApplication.run(SpringBootP02Application.class, args);
```

```
}  
  
}
```

@SpringBootApplication

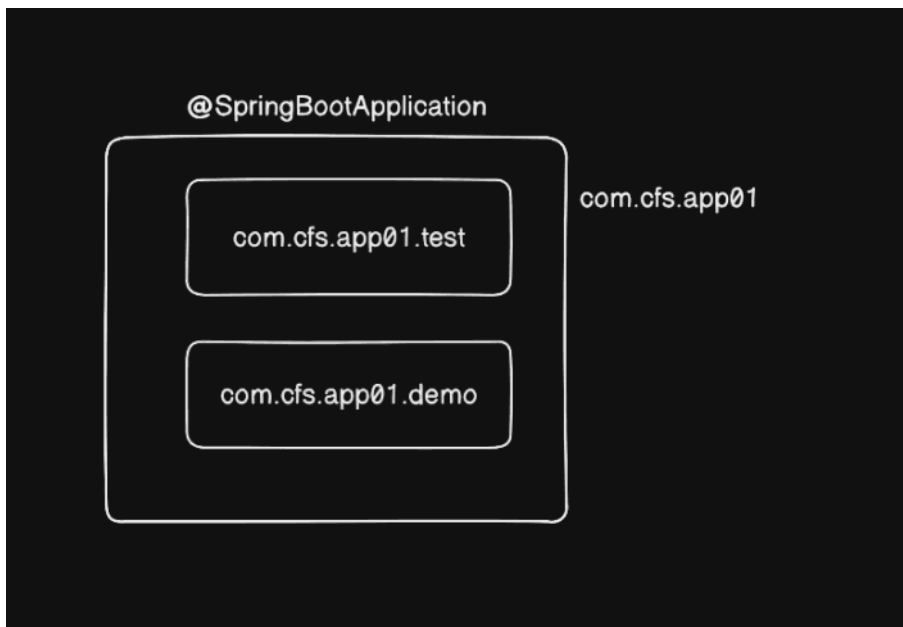
@SpringBootApplication is the main annotation placed on the starting class of a spring boot application it tells spring boot to start the spring container, apply auto-config and scan out project classes.

How it works:

1. Main() method calls SpringApplication.run()
2. Spring boot reads the class passed to run()
3. It see @SpringBootApplication
4. It treat the class as a configuration class ---?
5. It enable auto-config --- ?
6. It start component scanning from the package of that class ---?
7. Spring create beans and starts the embedded server.

Note:

Use it on the main class of almost every spring boot application. Keep this class in the root package so controller, service, repo and other packages are scanned automatically



@SpringBootApplication = configuration + auto-config + component scanning

@SpringBootApplication = @Configuration + @EnableAutoConfiguration + @ComponentScan

When we want our class to be become as spring bean in spring core ==> bean definition in xml
In spring boot we don't required xml config if we want to represent class as spring bean then we
Should use annotation

For example:

On top of the we can use ==>

@Controller

@Service

@Repository

Ex:

@Controller

```
public class StudentController {  
  
}
```

Annotation	Meaning
@Configuration	Class can defined beans
@EnableAutoConfiguration	Enable spring boot auto-config

Example:

spring-boot-starter-webmvc

Spring boot see that you are building a web application. So it automatically config things like:

1. Embedded tomcat
2. DispatcherServlet
3. Jackson

You don't have to configure all the required dependencies manually but without auto config we have to configure each dependency one by one.

How Auto-configuration works ?

Auto-config does not mean magic, it means spring boot writes common configuration for us based on condition.

Auto-configuration Internals

Spring boot does not blindly creates everything it checks conditions. If condition is true, configuration is applied.

If condition is false , configuration is skipped.

Component Scanning

Component scanning is the process where spring search our project and find classes marked with spring annotations after finding them, spring creates their objects and store them as beans inside IOC container.

Common annotation found by component scan:

1. @Component
2. @Service
3. @Repository
4. @Controller
5. @RestController
6. @Configuration

REST Controller

A REST controller is class that handles HTTP request and return HTTP response it is the entry point of the API

Request flow:

Client ---> embedded tomcat ---> DispatcherServlet ---> controller method ---> response

@RestController

Marks a class as REST API controller. It tells spring that methods in this class will handle we request and return data directly in the response body.

How it works:

1. Component scanning find the annotation @RestController class.
2. Spring create the bean
3. Mapping annotations like @GetMapping decide which method handle which URL
4. The method return value is written directly into http response
5. If the method returns the object , spring convert it into JSON

@RestController Vs @Controller

Use @RestController for backend REST APIs

Use @Controller when returning HTML pages with templates

@RestController = @Controller + @ResponseBody

@GetMapping

@GetMapping maps an HTTP GET request to a Java method, GET is used when the client want to read or fetch data.

Returning Object As JSON

Returning object as JSON means our controller returns a java object and spring boot converts into JSON before sending it to the client.

When to use :

1. Student details
2. Product Details
3. Order Details
4. Employe list

Important Note:

Jackson usually reads object data through getter. If getter are missing, JSON output may be empty on incomplete

Path Variable:

Path variable reads from the URL path, the value is part of the URL itself

Example:

URL: /student/101

Mapping :/student/{101}

Spring extracts 101 from the URL and assigns it to the method parameter id.

<http://localhost:8080/student/101>

```
@GetMapping("/student/{id}")
public String getStudentById(@PathVariable int id)
{
    return "Student id is "+id;
}
```

Request Parameter

Request parameter read value from the query string of the url
Query string starts after the ? Symbol

Example: /student/search?course=java

Spring read course = java and passes java into the method parameter

PathVariable Vs RequestParam

Both @PathVariable and @RequestParam read value from the Url the difference is where the value is present and why we using it.

How it works:

/student /101
101 is part of the path

@RequestParam
/student?id=101
101 is part of the query String

When to use:

Use @PathVariable when the value identifies one resource.
Use @RequestParam when the value filters, searchers, sort, customize the result

PathVariable ==> identifiers
RequestParam ==> filters

POST API and @RequestBody

POST is HTTP method mostly used to create new data
@RequestBody reads JSON from the HTTP request body and converts it into java object.

How it works:

1. Client send POST request with JSON body
2. Spring read the request body
3. Jackson convert JSON into java object
4. Controller method receives the object
5. Service layer can use that object to create data

Note: there is one annotation for each http method

Method	Annotation
--------	------------

GET	@GetMapping
POST	@PostMapping
DELETE	@DeleteMapping
PATCH	@PatchMapping
PUT	@PutMapping

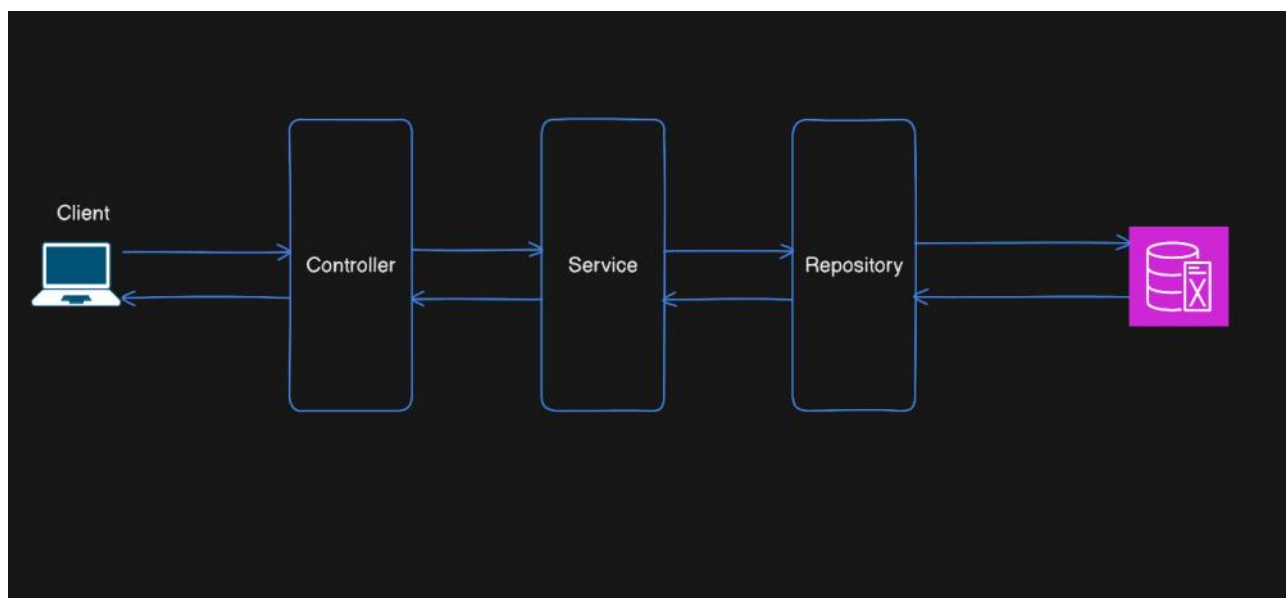
ResponseEntity

ResponseEntity represent the complete Http response. It allow us to control response body, status code, headers

How it works:

Without ResponseEntity, spring usually return 200 OK automatically, with ResponseEntity, we can return Correct status code like 201, 400, 404, 500

Spring boot layered Architecture



Mini Project

Student API without database

GET <http://localhost:8080/students>

GET <http://localhost:8080/students/101>

GET <https://localhost:8080/studnts/999>

Dependency Injection in Spring boot

Spring support mainly 3 type of common style:

1. Field Injection
2. Setter Injection
3. Constructor Injection

Field Injection

How it works:

1. Spring creates StudentController object
2. Spring create StudentService object
3. After object creation, Spring inject StudentService object into field of Controller

Problem:

1. Dependency is hidden
2. Field cannot be final

Note: Field Injection is easy to write but not the best professional style

Setter DI

Spring uses setter method for DI

Note: Setter injection is okay for optional dependencies. For required dependencies will constructor DI

Constructor DI

Parameter 1 of constructor in com.cfs.StudentAPI.controller.StudentController required a bean of type 'com.cfs.StudentAPI.service.CourseService' that could not be found.

Note: if a class has only one constructor @Autowire not required in spring boot

Note:

1. Field Injection is easy
2. Setter Injection is flexible
3. Constructor Injection is clean and recommended.

Which Autowiring Type is used in Each DI ?

Field DI, Setter DI, Constructor DI are DI types but Autowiring happens by type or by name.

Note:

@Autowired mainly resolves dependency by type.

If multiple beans of same type are found, then spring need extra help **@Qualifier, @Primary** or bean name

Field Injection Autowiring

Example:

```
@Autowired
private StudentService studentService;
```

How spring resolves:

1. Look at field type : StudentService
2. Service beans of StudentService type in container
3. If exactly one bean found, inject it
4. If multiple beans found, use @Qualifier/@Primary/name
5. If no bean found, error

Setter Injection Autowiring

```
@Autowired
public void setService(StudentService studentService) {
    this.studentService = studentService;
}
```

Constructor Injection Autowiring

```
@Autowired
public StudentController(StudentService studentService) {
    this.studentService = studentService;
}
```

Note: All the above DI are using by type.

Autowiring Resolution order for object

1. Spring checks required type
2. If one bean of that type exists, it inject it
3. If multiple bean exists, spring check @Qualifier

4. If Qualifier is not present, spring check @Primary
5. It still confused spring may use bean name/parameter name
6. If still not resolved, application failed

Annotation	Use
@Autowired	Ask spring to inject dependency
@Qualifier	Select specific bean by name
@Primary	Marks one bean as default

H.w : @Resource

Annotation Understanding

1. @Component : general-purpose spring bean
2. @Service : business logic
3. @Repository : database logic
4. @Controller :

Note: all these above annotation are type of @Component only

Reading Properties using @Value

@Value is used to inject a value from application .properties into java field , constructor, method parameter

Request Validation

Request validation checks incoming request data before it enter business logic it prevent invalid data from moving first layer to inside another layer.

Annotation:

1. @NotNull
2. @NotBlank
3. @NotEmpty
4. @Min
5. @Max
6. @Email
7. @Size

Interview Questions:

1. Why do we use spring boot instead of spring framework ?
2. What is auto-config in spring boot ?
3. What are starter dependencies ?
4. What is embedded server in spring boot and can we change it ?
5. What is `@SpringBootApplication`
6. Why should the main class be placed in root package ?
7. What is component Scanning ?
8. What is difference between `@Component` `@Service` `@Respository`
9. What is dependency Injection ?
10. Difference between `@Controller` & `@RestController` ?
11. Difference between `@PathVariable` and `@RequestParam` ?
12. How to resolve ambiguity in dependency ? ---> `@Qualifier` `@Primary` `@Resource`
13. How many types of scope we have for beans ?
14. What is prototype scope ?
15. How to change default port in spring boot ?
16. How to read properties from application.properties file